

# BCC202 - Estruturas de Dados I

## Aula 18: Filas de Prioridades

**Pedro Silva**

Universidade Federal de Ouro Preto, UFOP  
Departamento de Computação, DECOM  
Email: [silvap@ufop.edu.br](mailto:silvap@ufop.edu.br)



# Conteúdo

## Introdução

## Heap

Heap Mínimo e Heap Máximo

## Operações do Heap

Inserção

Remoção

Implementação

## Considerações Finais

## Bibliografia

# Introdução

Uma *fila de prioridade* é uma estrutura de dados para **manter um conjunto  $S$  de elementos**, cada qual com um valor associado denominado **chave** (Cormen et al., 2011).

Cada chave reflete a habilidade relativa de um item de **abandonar** o conjunto de itens rapidamente.

## Aplicações

SOs usam filas de prioridades, nas quais as chaves representam o tempo em que eventos devem ocorrer.

Métodos numérico iterativos são baseados na seleção repetida de um item com menor (maior) valor.

## Operações de uma Fila de Prioridade

1. **Constrói** uma fila de prioridades a partir de um conjunto com  $n$  itens.
2. Informa qual é o item com **maior prioridade** do conjunto.
3. Retira o item com **maior prioridade**.
4. **Insere** um novo item.
5. **Aumenta** a prioridade do item  $i$  para um novo valor que é maior que a prioridade do item atual.

## Operações de uma Fila de Prioridade

6. Substitui o item com **maior prioridade** por um novo item, a não ser que o novo item tenha mais prioridade.
7. Altera a prioridade de um item.
8. Remove um item qualquer.
9. Agrupar duas filas de prioridades em uma única.

**Lista linear ordenada:**

1. Constrói é  $O(n \log n)$  ou  $O(n^2)$ .
2. Insere é  $O(n)$ .
3. Retira é  $O(1)$ .
4. Altera é  $O(n)$ .

**Lista linear NÃO ordenada:**

1. Constrói é  $O(n)$ .
2. Insere é  $O(1)$ .
3. Retira é  $O(n)$ .
4. Altera é  $O(n)$ .

# Heap

## Heap

A melhor **representação de uma fila de prioridade** é por meio de uma estrutura de dados chamada **heap**:

- > Neste caso, Constrói é  $O(n)$ .
- > Insere, Retira, Substitui e Altera são  $O(\log n)$ .

### Observação sobre a função Agrupar:

Para implementar a operação **Agrupar** de forma eficiente e ainda preservar um custo logarítmico para as operações **Inserir**, **Retira**, **Substitui** e **Altera** é necessário utilizar estruturas de dados mais sofisticadas, tais como árvores binomiais (Vuillemin, 1978).

O que é *heap* afinal?

## O que é *heap* afinal?

Um *heap* é um **vetor** que simula uma **árvore binária quase completa**.

## O que é *heap* afinal?

Um *heap* é um **vetor** que simula uma **árvore binária quase completa**.

## Mas o que é uma árvore binária?

## O que é *heap* afinal?

Um *heap* é um **vetor** que simula uma **árvore binária quase completa**.

## Mas o que é uma árvore binária?

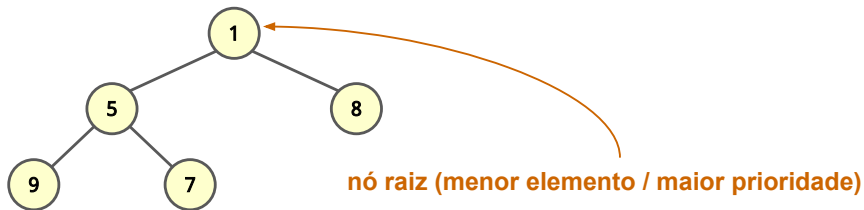
*Árvore binária é uma estrutura de dados caracterizada por: (i) ou não tem elemento algum (árvore vazia); (ii) ou tem um elemento distinto, denominado raiz, com dois apontamentos para duas estruturas diferentes, denominadas sub-árvore esquerda e sub-árvore direita*<sup>1</sup>.

---

<sup>1</sup>[https://pt.wikibooks.org/wiki/Algoritmos\\_e\\_Estruturas\\_de\\_DadosÁrvores\\_Binárias](https://pt.wikibooks.org/wiki/Algoritmos_e_Estruturas_de_DadosÁrvores_Binárias)

## Heap

- ▶ É uma árvore binária em que um nó filho é sempre **maior ou igual** a um nó pai.
- ▶ Ou seja:  $chave(v) \geq chave(pai(v))$ .



O que é uma árvore binária completa?

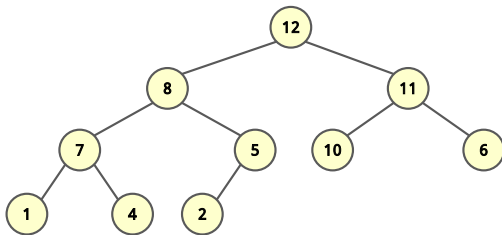
## Árvore binária completa:

- ▶ Os nós são numerados de 0 a  $n - 1$ .
- ▶ O primeiro nó é chamado raiz.
- ▶ O nó  $\frac{k-1}{2}$  é o pai do nó  $k$ , para  $0 < k < n$ .
- ▶ Os nós  $2k+1$  e  $2k+2$  são os filhos à esquerda e à direita do nó  $k$ , para  $0 < k \leq \frac{n}{2}$ .

## Heap vs. Árvore Binária

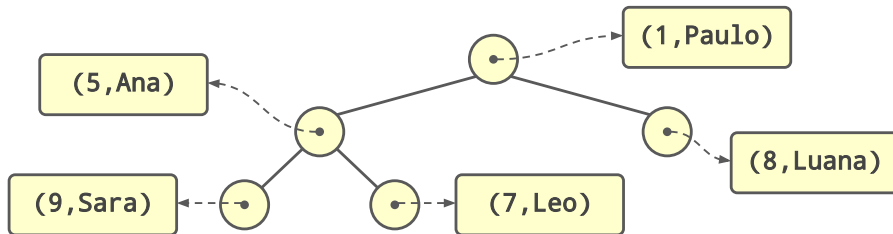
Um **heap** é idealizado como uma **árvore binária quase completa**: cada **nó folha** da **árvore** está no **nível N-1** ou **N-2** - considerando uma árvore de níveis  $0..N-1$ .

- ▶ as folhas são identificadas da esquerda para a direita
- ▶ a prioridade associada ao nó pai é maior do que as associadas ao(s) seu(s) filho(s)
- ▶ quando dois filhos, não há uma ordenação entre eles



## Representação de um Heap

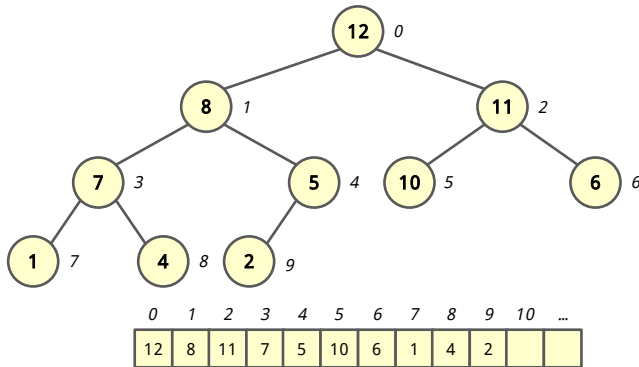
- ▶ Cada nó possui uma tupla (chave, elemento).
- ▶ Assim, cada nó do heap armazena todo o item



Um **heap** é um **vetor** que simula uma **árvore binária**.

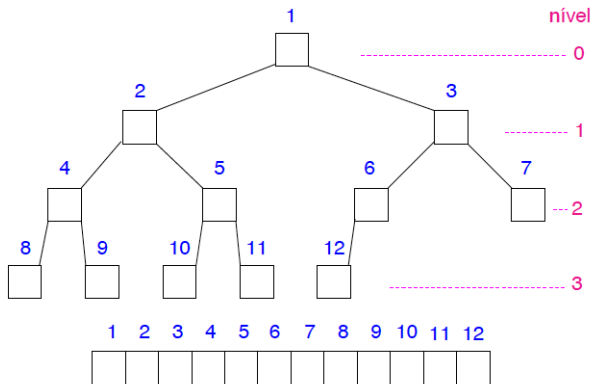
Numeramos os nós da árvore de **cima para baixo e da esquerda para direita** em cada nível.

- ▶ A árvore está completamente preenchida em todos os níveis, exceto possivelmente no último nível, ou nível mais baixo ( $N - 1$ ), preenchido a partir da esquerda até um



## Níveis

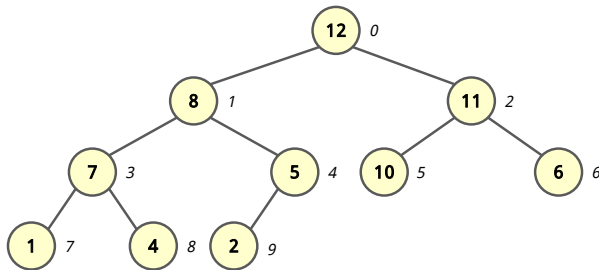
A **raiz** da árvore está no **nível zero (N0)**.



## Posição de um Nó no Vetor - Pai e Filho(s) de um nó

Considere um vetor  $A[0...n-1]$  representando um **heap**.

- ▶ Cada posição do vetor corresponde a um nó do **heap**.
  - ▶ O **pai** de um nó  $i$  é  $\lfloor (i-1)/2 \rfloor$  para  $0 < i < n$  (o nó **0** não tem pai).
  - ▶ O(s) **filho(s)** de um nó  $i$ , para  $0 \leq i < \lfloor n/2 \rfloor$ :
    - ▶ Seu **filho à esquerda** está na posição  $2*i+1$ .
    - ▶ Seu **filho à direita** está na posição  $2*i+2$ .

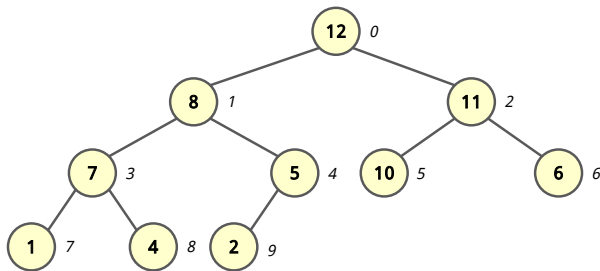


## Posição de um Nó no Vetor - Nós folhas

Considere um vetor  $A[0 \dots n-1]$  representando um **heap**.

Os **nós folhas** da *árvore binária* estão nas posições do vetor:

►  $\lfloor n/2 \rfloor < i \leq n-1$



## Heaps

- ▶ As chaves na árvore satisfazem a condição do heap.
- ▶ A chave em cada nó tem **mais prioridade** do que as chaves em seus filhos.
- ▶ A chave no nó raiz é a chave com **maior prioridade** do conjunto.
- ▶ Como dito, uma árvore binária completa pode ser representada por um array.

## Heaps

- ▶ A representação é extremamente compacta.
- ▶ Permite caminhar pelos nós da árvore facilmente.
  - ▶ Os filhos de um nó  $i$  estão nas posições  $2i + 1$  e  $2i + 2$ .
  - ▶ O pai de um nó  $i$  está na posição  $(i - 1)/2$ .
- ▶ Na representação do heap em um vetor, a chave **com maior prioridade** está sempre na posição 0 deste vetor.

## Construção de um Heap

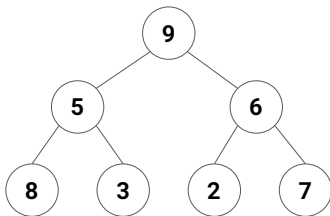
- ▶ Um algoritmo elegante para construir o heap foi proposto por Floyd em 1964.
- ▶ O algoritmo não necessita de nenhuma memória auxiliar.
- ▶ Dado um vetor  $A[0], A[1], \dots, A[n-1]$ :
  - ▶ Os itens  $A[n/2], A[n/2+1], \dots, A[n-1]$  formam um heap válido pois são nós folhas (nós que não possuem filhos).
  - ▶ Neste intervalo não existem dois índices  $i$  e  $j$  tais que  $j = 2i + 1$  ou  $j = 2i + 2$ .

## Exemplo da construção de um Heap

| 0 | 1 | 2 | 3 | 4 | 5 | 6 |
|---|---|---|---|---|---|---|
| 9 | 5 | 6 | 8 | 3 | 2 | 7 |

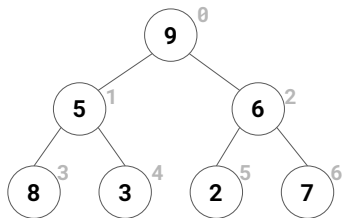
## Exemplo da construção de um Heap

| 0 | 1 | 2 | 3 | 4 | 5 | 6 |
|---|---|---|---|---|---|---|
| 9 | 5 | 6 | 8 | 3 | 2 | 7 |



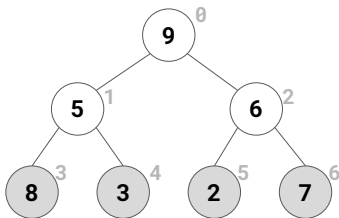
## Exemplo da construção de um Heap

| 0 | 1 | 2 | 3 | 4 | 5 | 6 |
|---|---|---|---|---|---|---|
| 9 | 5 | 6 | 8 | 3 | 2 | 7 |



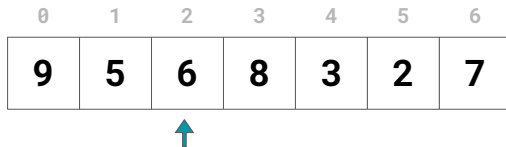
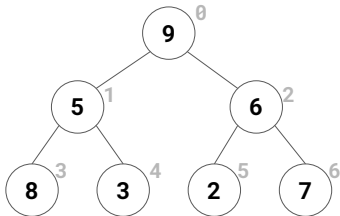
## Exemplo da construção de um Heap

| 0 | 1 | 2 | 3 | 4 | 5 | 6 |
|---|---|---|---|---|---|---|
| 9 | 5 | 6 | 8 | 3 | 2 | 7 |



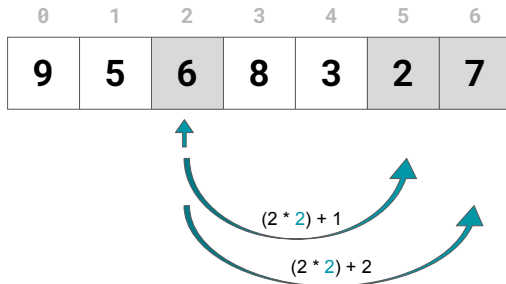
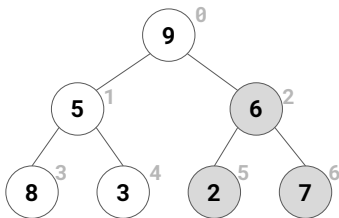
## Exemplo da construção de um Heap

Esq = 2



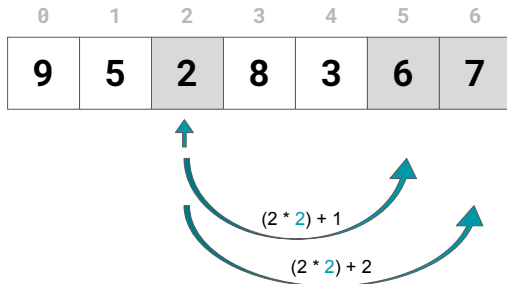
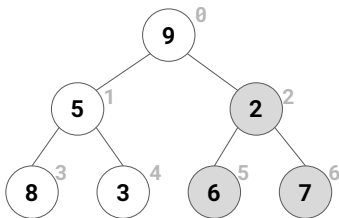
## Exemplo da construção de um Heap

Esq = 2



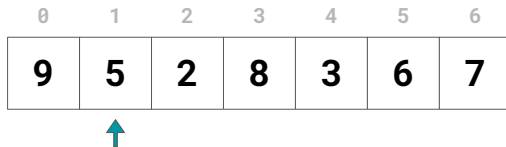
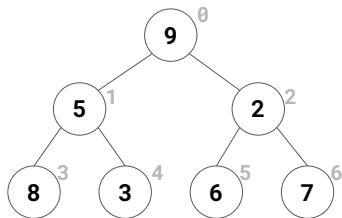
## Exemplo da construção de um Heap

Esq = 2



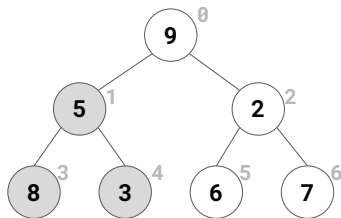
## Exemplo da construção de um Heap

Esq = 1

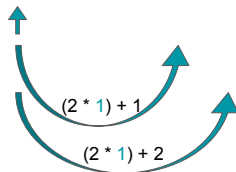


## Exemplo da construção de um Heap

Esq = 1

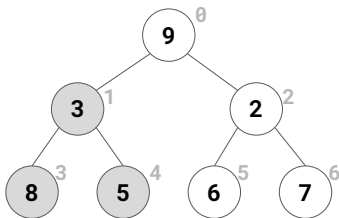


| 0 | 1 | 2 | 3 | 4 | 5 | 6 |
|---|---|---|---|---|---|---|
| 9 | 5 | 2 | 8 | 3 | 6 | 7 |

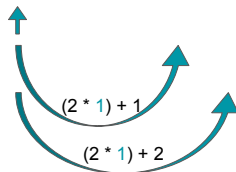


## Exemplo da construção de um Heap

Esq = 1

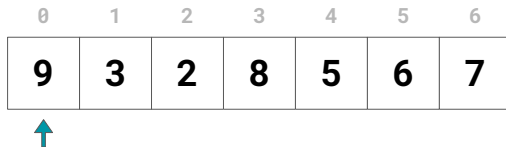
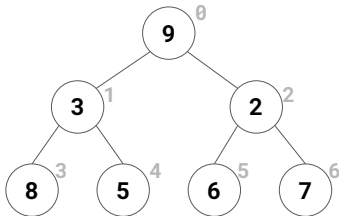


| 0 | 1 | 2 | 3 | 4 | 5 | 6 |
|---|---|---|---|---|---|---|
| 9 | 3 | 2 | 8 | 5 | 6 | 7 |



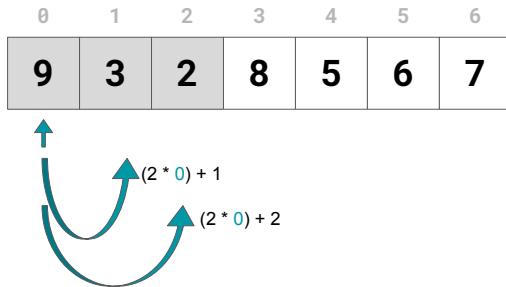
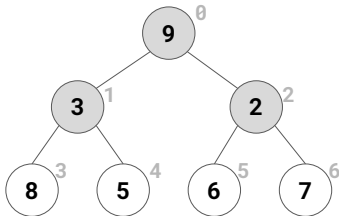
## Exemplo da construção de um Heap

Esq = 0



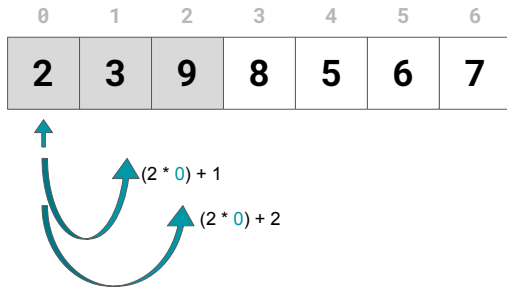
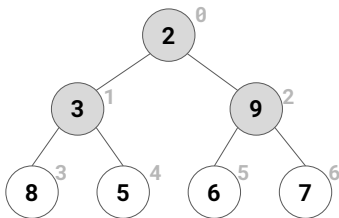
## Exemplo da construção de um Heap

Esq = 0



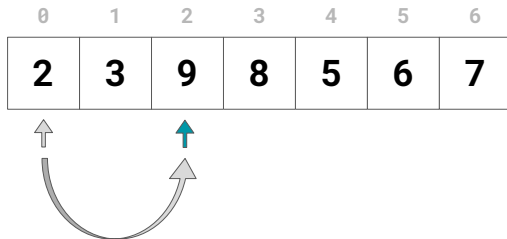
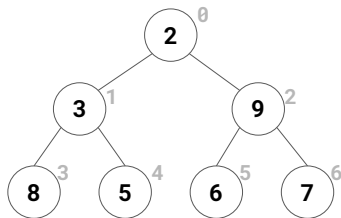
## Exemplo da construção de um Heap

Esq = 0



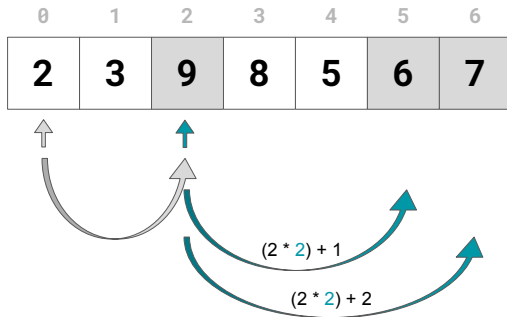
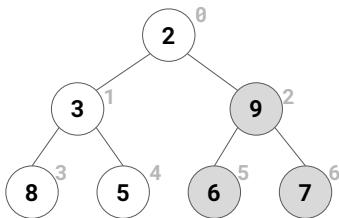
## Exemplo da construção de um Heap

Esq = 0

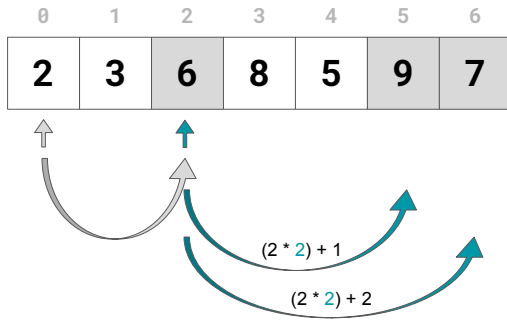
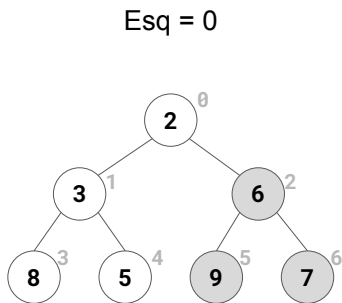


## Exemplo da construção de um Heap

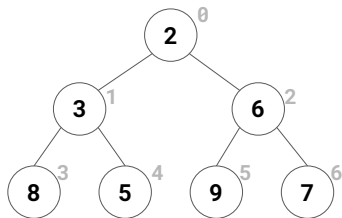
Esq = 0



## Exemplo da construção de um Heap



## Exemplo da construção de um Heap



| 0 | 1 | 2 | 3 | 4 | 5 | 6 |
|---|---|---|---|---|---|---|
| 2 | 3 | 6 | 8 | 5 | 9 | 7 |

## Considerações sobre o exemplo

- ▶ Os itens de  $A[3]$  a  $A[6]$  formam um heap.
- ▶ O heap é estendido para a esquerda, englobando o item  $A[2]$ , pai dos itens  $A[5]$  e  $A[6]$ .
- ▶ A condição de heap é violada:
  - ▶ O heap é refeito trocando os itens  $A[2]$  e  $A[5]$ .
- ▶ O item 5 ( $A[1]$ ) é incluindo no heap ( $\text{Esq} = 1$ ).
- ▶ A condição de heap é violada:
  - ▶ O heap é refeito trocando os itens  $A[1]$  e  $A[4]$ .

## Considerações sobre o exemplo

- ▶ O item 9 ( $A[0]$ ) é incluindo no heap ( $Esq = 0$ ).
- ▶ A condição de heap é violada:
  - ▶ O heap é refeito trocando os itens  $A[0]$  e  $A[1]$ .
- ▶ Como a condição ainda está sendo violada:
  - ▶ O heap é refeito trocando os itens  $A[1]$  e  $A[4]$ .
- ▶ Como resultado, o heap foi construído:

2   3   6   8   5   9   7

## Heap Mínimo ou Mini-Heap

Para todo nó  $i$  tal que  $0 < i \leq n - 1$ :

- ▶  $A[\lfloor (i - 1)/2 \rfloor] \leq A[i]$ .
- ▶ **O menor elemento está na raiz.**
- ▶ Os valores dos nós satisfazem a **propriedade de heap mínimo**.

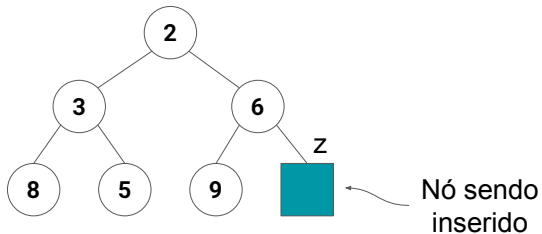
## Heap Máximo ou Max-Heap

Para todo nó  $i$  tal que  $0 < i \leq n - 1$ :

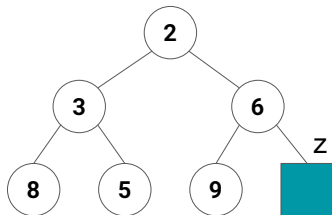
- ▶  $A[\lfloor (i - 1)/2 \rfloor] \geq A[i]$ .
- ▶ **O maior elemento está na raiz.**
- ▶ Os valores dos nós satisfazem a **propriedade de heap máximo**.

## Operações do Heap

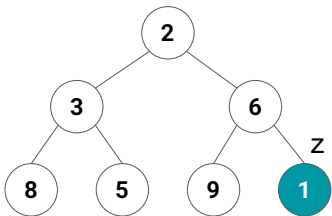
## Inserir um Nó no Heap



## Inserir um Nó no Heap

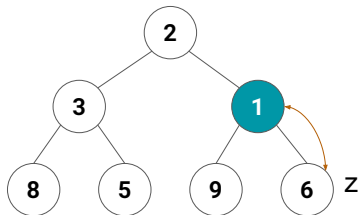


Nó sendo  
inserido

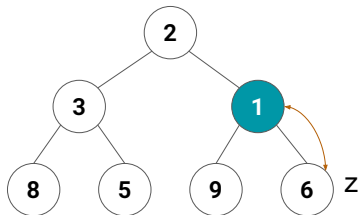


Comparar o nó inserido com os pais e trocar enquanto ele for menor que o pai ou até que ele seja o nó raiz

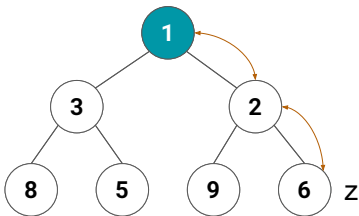
## Inserir um Nó no Heap



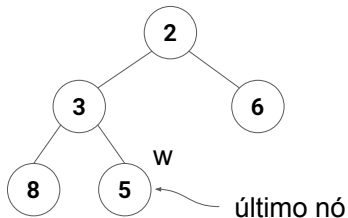
## Inserir um Nó no Heap



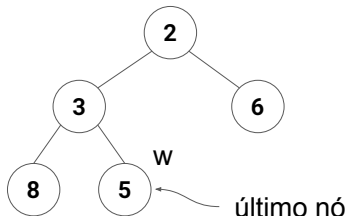
Na pior das hipóteses o custo de uma inserção será  $O(\log n)$ , equivalente à altura da árvore



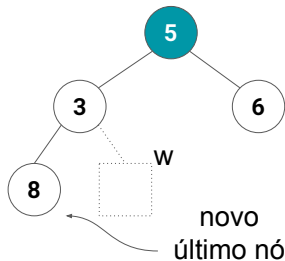
## Remover um Nó do Heap



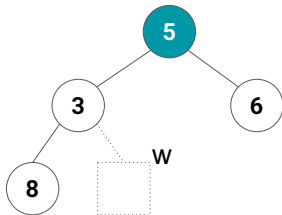
## Remover um Nó do Heap



Trocar o nó raiz pelo  
último nó do heap e  
remover o último nó

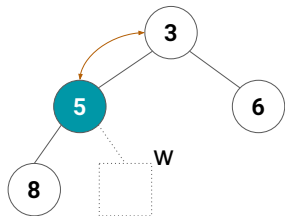
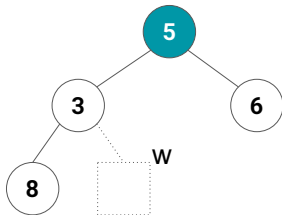


## Remover um Nó do Heap



Refazer o heap!

## Remover um Nó do Heap



Refazer o heap!

Na pior das hipóteses o custo de uma remoção será  $O(\log n)$ , equivalente à altura da árvore

---

---

**1 Algorithm: HEAP\_CONSTROI****Input:** int\* v, int n**2 begin****3**      $esq \leftarrow (n/2) - 1$  // *esq = primeiro no antes do no folha do heap***4**     **while**  $esq \geq 0$  **do****5**         HEAP\_REFAZ(v, esq, n - 1)**6**          $esq --$ **7**     **end****8 end**

---

---

**1 Algorithm: HEAP\_REFAZ****Input:** int\* v, int esq, int dir**2 begin****3**      $i \leftarrow \text{esq}$ **4**      $j \leftarrow i * 2 + 1$ **5**      $\text{aux} \leftarrow v[i]$ **6**     **while**  $j \leq \text{dir}$  **do****7**         **if**  $j < \text{dir}$  **and**  $v[j] < v[j + 1]$  **then****8**              $j \leftarrow j + 1$  *// j recebe o outro filho de i***9**         **end****10**        **if**  $\text{aux} \geq v[j]$  **then****11**             $\text{break}$ **12**        **end****13**         $v[i] \leftarrow v[j]$ **14**         $i \leftarrow j$ **15**         $j \leftarrow i * 2 + 1$ **16**     **end****17**      $v[i] \leftarrow \text{aux}$ **18 end**

---

## Considerações Finais

## Conclusão

- ▶ Introdução das filas de prioridades e árvores binárias.
- ▶ Custo de construção  $O(n)$ , inserção e remoção de  $O(\log n)$ .
- ▶ Introdução outra técnica de projeto de algoritmos: a utilização de uma estrutura de dados para gerenciar informações.

## *Heap Sort*

## Bibliografia

## Bibliografia

Os conteúdos deste material, incluindo figuras, textos e códigos, foram extraídos ou adaptados de:

-  Cormen, Thomas H. and Leiserson, Charles E. and Rivest, Ronald L. and Stein, Clifford.  
*Introduction to Algorithms.*  
The MIT Press, 2011.

## Exercício

- ▶ Dada a sequência de números: 3 4 0 2 5 1 3.
- ▶ Construa um *Heap* (em que o maior elemento fica na raiz), passo-a-passo. Em seguida, remova os 3 número prioritários.